

# Scaler

## Business Case Study

### Topic: SQL

### Target

Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.

By analysing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

### Analysis of the Target's Data in Brazil region between the year 2016 and 2018

**Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**

1. Data type of all columns in the "customers" table.

Answer:

Query:

```
SELECT
    column_name,
    data_type
FROM
    `new-project-prathik.Target_Ecommerce.INFORMATION_SCHEMA.COLUMN`
WHERE
    table_name = 'customers';
```

Output:

| Query results   |                          |           |       |                 |                   |
|-----------------|--------------------------|-----------|-------|-----------------|-------------------|
| JOB INFORMATION |                          | RESULTS   | CHART | JSON            | EXECUTION DETAILS |
|                 |                          |           |       | EXECUTION GRAPH |                   |
| Row             | column_name              | data_type |       |                 |                   |
| 1               | customer_id              | STRING    |       |                 |                   |
| 2               | customer_unique_id       | STRING    |       |                 |                   |
| 3               | customer_zip_code_prefix | INT64     |       |                 |                   |
| 4               | customer_city            | STRING    |       |                 |                   |
| 5               | customer_state           | STRING    |       |                 |                   |

### Insights:

- There are 5 columns in the table customers and except for customer\_zip\_code column all other column have STRING datatype and zip\_code has INT datatype.

Recommendations: NA

- Get the time range between which the orders were placed.

### Answer:

#### Query:

```
SELECT
  Count(order_id) AS No_of_orders,
  MIN(order_purchase_timestamp) AS First_order_date,
  MAX(order_purchase_timestamp) AS Last_order_date,
  DATE_DIFF(MAX(order_purchase_timestamp) , MIN(order_purchase_timestamp) , DAY) Total_interval,
FROM
  Target_Ecommerce.orders
```

#### Output:

| Query results   |              |                         |                         |                |                   | <a href="#">SAVE RESULTS</a> |
|-----------------|--------------|-------------------------|-------------------------|----------------|-------------------|------------------------------|
| JOB INFORMATION |              | RESULTS                 | CHART                   | JSON           | EXECUTION DETAILS | EXECUTION GRAPH              |
| Row             | No_of_orders | First_order_date        | Last_order_date         | Total_interval |                   |                              |
| 1               | 99441        | 2016-09-04 21:15:19 UTC | 2018-10-17 17:30:18 UTC | 772            |                   |                              |

### Insights:

- In the given data set the first order and the last order was placed on 4<sup>th</sup> Sep 2016 and 17<sup>th</sup> Oct 2018 respectively.
- Total 99,441 orders were placed in a time period of 772 days.

Recommendations: NA

- Count the Cities & States of customers who ordered during the given period.

### Answer:

#### Query:

```
SELECT
  EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
  COUNT(DISTINCT customer_state) AS No_of_States,
  COUNT(DISTINCT customer_city) AS No_of_Cities
FROM
```

```

Target_Ecommerce.orders o
JOIN
Target_Ecommerce.customers c ON o.customer_id = c.customer_id
GROUP BY
Year
ORDER BY
Year;

```

*Output:*

| JOB INFORMATION |        | RESULTS        | CHART          | JSON |
|-----------------|--------|----------------|----------------|------|
| Row             | Year ▼ | No_of_States ▼ | No_of_Cities ▼ |      |
| 1               | 2016   | 21             | 179            |      |
| 2               | 2017   | 27             | 3266           |      |
| 3               | 2018   | 27             | 3252           |      |

*Insights:*

- In 2016 orders came from 21 states and 179 cities, means orders came from most of the states, but not all cities were involved.
- In 2017 and 2018 orders were made from all states and more than 55% of the cities.

*Recommendations:*

- To reach out to remaining cities we can give advertisements in the public places.

### **In-depth Exploration:**

4. Is there a growing trend in the no. of orders placed over the past years?

Answer:

*Query:*

```

WITH
test1 AS
(
SELECT
EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
EXTRACT(Month FROM order_purchase_timestamp) AS Month,
COUNT(DISTINCT order_id) AS Current_Month_orders
FROM
Target_Ecommerce.orders o
LEFT JOIN
Target_Ecommerce.customers c ON o.customer_id = c.customer_id
LEFT JOIN

```

```

Target_Ecommerce.geolocation g ON c.customer_zip_code_prefix = g.geolocation_zip_code_prefix
GROUP BY
    Year,Month
)

```

```

SELECT
    *,
    ROUND(((Current_Month_orders-Previous_Month_orders) / Previous_Month_orders) * 100, 2)
    AS Percentage_Change
FROM
    (
        SELECT
            Year,
            Month,
            Lead(Current_Month_orders) OVER(ORDER BY Year DESC, Month DESC) AS Previous_Month_orders,
            Current_Month_orders
        FROM
            test1
    ) t
Order by
    Year, Month;

```

Output:

| JOB INFORMATION |        | RESULTS |                       | CHART                | JSON              | EXECUTION DETAILS |
|-----------------|--------|---------|-----------------------|----------------------|-------------------|-------------------|
| Row             | Year ▼ | Month   | Previous_Month_orders | Current_Month_orders | Percentage_Change |                   |
| 1               | 2016   | 9       | <i>null</i>           | 4                    | <i>null</i>       |                   |
| 2               | 2016   | 10      | 4                     | 324                  | 8000.0            |                   |
| 3               | 2016   | 12      | 324                   | 1                    | -99.69            |                   |
| 4               | 2017   | 1       | 1                     | 800                  | 79900.0           |                   |
| 5               | 2017   | 2       | 800                   | 1780                 | 122.5             |                   |
| 6               | 2017   | 3       | 1780                  | 2682                 | 50.67             |                   |
| 7               | 2017   | 4       | 2682                  | 2404                 | -10.37            |                   |
| 8               | 2017   | 5       | 2404                  | 3700                 | 53.91             |                   |
| 9               | 2017   | 6       | 3700                  | 3245                 | -12.3             |                   |
| 10              | 2017   | 7       | 3245                  | 4026                 | 24.07             |                   |
| 11              | 2017   | 8       | 4026                  | 4331                 | 7.58              |                   |
| 12              | 2017   | 9       | 4331                  | 4285                 | -1.06             |                   |
| 13              | 2017   | 10      | 4285                  | 4631                 | 8.07              |                   |

Insights:

- Starting from Sep 2016 to Apr 2018 there is a growing trend, there are dips in the number of orders in between, but the number of orders had clocked to 7,000 from being in 1,000.

- But from Apr 2018 we can see a regular decrease in the number of orders placed, and it as come down to 16 in the month of Sep with a sudden dip(May be there is a issue with the data as we can only observe a slight ups and down in the previous months)

*Recommendations:*

- Check the data of Sep and Oct month for the year 2018 to verify the data is correct.
- If it is true then we need see why there is sudden drastic dip and take actions accordingly.

5. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

Answer:

*Query:*

```

WITH
test1 AS
(
    SELECT
        EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
        EXTRACT(Month FROM order_purchase_timestamp) AS Month,
        COUNT(DISTINCT order_id) AS No_of_orders
    FROM
        Target_Ecommerce.orders o
    LEFT JOIN
        Target_Ecommerce.customers c ON o.customer_id = c.customer_id
    LEFT JOIN
        Target_Ecommerce.geolocation g ON c.customer_zip_code_prefix = g.geolocation_zip_code_prefix
    GROUP BY
        Year,Month
)

SELECT
    Year,
    Month,
    No_of_orders,
    Round ( SUM (No_of_orders) OVER ( PARTITION BY Month ORDER BY Month DESC), 0) AS Monthly_Avg
FROM
    test1
ORDER BY
    Month, Year;

```

Output:

## Query results

| JOB INFORMATION |        | RESULTS | CHART          | JSON          | EXECUTION DETAILS |
|-----------------|--------|---------|----------------|---------------|-------------------|
| Row             | Year ▼ | Month ▼ | No_of_orders ▼ | Monthly_Avg ▼ |                   |
| 1               | 2017   | 1       | 800            | 8069.0        |                   |
| 2               | 2018   | 1       | 7269           | 8069.0        |                   |
| 3               | 2017   | 2       | 1780           | 8508.0        |                   |
| 4               | 2018   | 2       | 6728           | 8508.0        |                   |
| 5               | 2017   | 3       | 2682           | 9893.0        |                   |
| 6               | 2018   | 3       | 7211           | 9893.0        |                   |
| 7               | 2017   | 4       | 2404           | 9343.0        |                   |
| 8               | 2018   | 4       | 6939           | 9343.0        |                   |
| 9               | 2017   | 5       | 3700           | 10573.0       |                   |
| 10              | 2018   | 5       | 6873           | 10573.0       |                   |
| 11              | 2017   | 6       | 3245           | 9412.0        |                   |

Insights:

- Average number of orders in the month May, Jul, Aug are high compared to other months.
- Number of orders placed in Nov and Dec are low.
- Other months have good average number of orders.

Recommendations:

- We can provide quarterly or yearly offers like year-end-sale to clear up the inventory stocks.

6. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
- 0-6 hrs: Dawn
  - 7-12 hrs: Mornings
  - 13-18 hrs: Afternoon
  - 19-23 hrs: Night

Answer:

Query:

```
WITH
test1 AS
(
  SELECT
    EXTRACT(HOUR FROM order_purchase_timestamp) AS Hour
  FROM Target_Ecommerce.orders
)

SELECT
CASE
  WHEN Hour IN (0,1,2,3,4,5) THEN 'Dawn'
```

```

        WHEN Hour IN (6,7,8,9,10,11) THEN 'Morning'
        WHEN Hour IN (12,13,14,15,16,17) THEN 'Afternoon'
        ELSE 'Night'
    END AS Interval1,
    COUNT(*) AS Total_orders
FROM
    test1
GROUP BY
    Interval1
ORDER BY
    Total_orders DESC;

```

Output:

| JOB INFORMATION |             | RESULTS        | CHART | JSOI |
|-----------------|-------------|----------------|-------|------|
| Row             | Interval1 ▼ | Total_orders ▼ |       |      |
| 1               | Afternoon   | 38361          |       |      |
| 2               | Night       | 34100          |       |      |
| 3               | Morning     | 22240          |       |      |
| 4               | Dawn        | 4740           |       |      |

Insights:

- We can see most orders are placed during after afternoon and night, which means users are mostly active during this period.
- Highest number of orders came in the afternoon.

Recommendations:

- We can give calculated discounts and offers during morning to increase the sales in the that period.

---

### Evolution of E-commerce orders in the Brazil region:

7. Get the month-on-month no. of orders placed in each state.

Answer:

Query:

```

WITH
    test1 AS
    (
        SELECT
            c.customer_state AS State,
            EXTRACT(Month FROM order_purchase_timestamp) AS Month,
            COUNT(DISTINCT order_id) AS No_of_orders
        FROM

```

```

    Target_Ecommerce.orders o
JOIN
    Target_Ecommerce.customers c ON o.customer_id = c.customer_id
GROUP BY
    State,Month
)

SELECT
    State,
    Month,
    No_of_orders,
    SUM(No_of_orders) OVER(PARTITION BY STATE) AS Total_orders_in_state
FROM
    test1
ORDER BY
    State, Month;

```

*Output:*

| JOB INFORMATION |       | RESULTS | CHART        | JSON                  | EXECUTION DETAILS |
|-----------------|-------|---------|--------------|-----------------------|-------------------|
| Row             | State | Month   | No_of_orders | Total_orders_in_state |                   |
| 1               | AC    | 1       | 8            | 81                    |                   |
| 2               | AC    | 2       | 6            | 81                    |                   |
| 3               | AC    | 3       | 4            | 81                    |                   |
| 4               | AC    | 4       | 9            | 81                    |                   |
| 5               | AC    | 5       | 10           | 81                    |                   |
| 6               | AC    | 6       | 7            | 81                    |                   |
| 7               | AC    | 7       | 9            | 81                    |                   |
| 8               | AC    | 8       | 7            | 81                    |                   |
| 9               | AC    | 9       | 5            | 81                    |                   |
| 10              | AC    | 10      | 6            | 81                    |                   |
| 11              | AC    | 11      | 5            | 81                    |                   |

*Insights:*

- We can month-on-month number of orders have increase in states like SP, RJ and MJ and almost constant in few states.

*Recommendations:* NA

8. How are the customers distributed across all the states?

Answer:

*Query:*

```
WITH
test1 AS
(
SELECT
customer_state AS State,
COUNT(DISTINCT c.customer_unique_id) AS No_of_customers,
COUNT(DISTINCT o.order_id) AS No_of_orders
FROM
Target_Ecommerce.customers c
LEFT JOIN Target_Ecommerce.orders o ON c.customer_id = o.customer_id
GROUP BY
State
)

SELECT
*,
ROUND(((No_of_orders - No_of_customers) / No_of_customers) * 100 , 1) AS Repeat_percentage
FROM test1
ORDER BY No_of_customers DESC;
```

*Output:*

| JOB INFORMATION |       | RESULTS         | CHART        | JSON              | EXECUTION DET. |
|-----------------|-------|-----------------|--------------|-------------------|----------------|
| Row             | State | No_of_customers | No_of_orders | Repeat_percentage |                |
| 1               | SP    | 40302           | 41746        | 3.6               |                |
| 2               | RJ    | 12384           | 12852        | 3.8               |                |
| 3               | MG    | 11259           | 11635        | 3.3               |                |
| 4               | RS    | 5277            | 5466         | 3.6               |                |
| 5               | PR    | 4882            | 5045         | 3.3               |                |
| 6               | SC    | 3534            | 3637         | 2.9               |                |
| 7               | BA    | 3277            | 3380         | 3.1               |                |
| 8               | DF    | 2075            | 2140         | 3.1               |                |
| 9               | ES    | 1964            | 2033         | 3.5               |                |
| 10              | GO    | 1952            | 2020         | 3.5               |                |
| 11              | PE    | 1609            | 1652         | 2.7               |                |
| 12              | CE    | 1313            | 1336         | 1.8               |                |
| 13              | PA    | 949             | 975          | 2.7               |                |

*Insights:*

- SP, RJ, MG, RS and PR are top 5 highest orders states and also generating highest revenue.
- Even after having larger number customers in these region the repeat rate of customer is almost same to other states, which is around 3.5 on an average.

*Recommendations:*

- To increase the repeat rate of customers we can send text messages with offers to their cell phone.
  - Make sure they have a good customer experience when they are shopping by assisting them during the process.
  - Memberships to new customer to increase the customer data and to be in touch with the customers with small text messages.
- 

**Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.**

9. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment value" column in the payments table to get the cost of orders.

Answer:

*Query:*

```
WITH
test1 AS
(
SELECT
    EXTRACT(YEAR FROM order_purchase_timestamp) AS Year,
    EXTRACT(Month FROM order_purchase_timestamp) AS Month,
    ROUND(SUM(payment_value), 2) AS Current_Month_Cost_of_orders
FROM
    Target_Ecommerce.orders o
JOIN
    Target_Ecommerce.payments p ON o.order_id = p.order_id

GROUP BY
    Year, Month
),

test2 AS
(SELECT
    *,
    LAG(Current_Month_Cost_of_orders, 1) OVER(ORDER BY Year, Month ) AS
Previous_Month_Cost_of_orders
FROM
    test1
)

SELECT
    *,
```

```

ROUND(((Current_Month_Cost_of_orders - Previous_Month_Cost_of_orders) /
Previous_Month_Cost_of_orders) * 100 , 2) AS Monthly_Percentage_Change
FROM
test2
WHERE
Year IN (2017,2018)
AND
Month Between 1 and 8
ORDER BY
Year DESC, Month DESC;

```

*Output:*

| JOB INFORMATION |      | RESULTS | CHART                        | JSON                          | EXECUTION DETAILS         | EXECUTION GRAPH |
|-----------------|------|---------|------------------------------|-------------------------------|---------------------------|-----------------|
| Row             | Year | Month   | Current_Month_Cost_of_orders | Previous_Month_Cost_of_orders | Monthly_Percentage_Change |                 |
| 1               | 2018 | 8       | 1022425.32                   | 1066540.75                    | -4.14                     |                 |
| 2               | 2018 | 7       | 1066540.75                   | 1023880.5                     | 4.17                      |                 |
| 3               | 2018 | 6       | 1023880.5                    | 1153982.15                    | -11.27                    |                 |
| 4               | 2018 | 5       | 1153982.15                   | 1160785.48                    | -0.59                     |                 |
| 5               | 2018 | 4       | 1160785.48                   | 1159652.12                    | 0.1                       |                 |
| 6               | 2018 | 3       | 1159652.12                   | 992463.34                     | 16.85                     |                 |
| 7               | 2018 | 2       | 992463.34                    | 1115004.18                    | -10.99                    |                 |
| 8               | 2018 | 1       | 1115004.18                   | 878401.48                     | 26.94                     |                 |
| 9               | 2017 | 8       | 674396.32                    | 592382.92                     | 13.84                     |                 |
| 10              | 2017 | 7       | 592382.92                    | 511276.38                     | 15.86                     |                 |
| 11              | 2017 | 6       | 511276.38                    | 592918.82                     | -13.77                    |                 |
| 12              | 2017 | 5       | 592918.82                    | 417788.03                     | 41.92                     |                 |
| 13              | 2017 | 4       | 417788.03                    | 449863.6                      | -7.13                     |                 |

*Insights:*

- In Feb 2017 we can see the 110% of increase in the cost of orders, apart from this we can see Mar and May in 2017 have 54% and 41% increase respectively.
- During the year 2017 we can see all most all month having at least 10% increase in the cost of orders.
- In the year 2018 we can observe the dips in the % of cost of orders.

*Recommendations:*

- We can try giving combo offers to increase the cost of orders.

10. Calculate the Total & Average value of order price for each state.

Answer:

*Query:*

```

SELECT
customer_state AS State,
ROUND(SUM(price), 2) AS Total_order_price,
ROUND(AVG(price) , 2) AS Avg_order_price
FROM

```

```

Target_Ecommerce.order_items o1
JOIN
Target_Ecommerce.orders o2 ON o1.order_id = o2.order_id
JOIN
Target_Ecommerce.customers c ON o2.customer_id = c.customer_id
GROUP BY
State
ORDER BY
,Avg_order_price, Total_order_price;

```

Output:

| JOB INFORMATION |         | RESULTS           | CHART           | JSON | EXECUTION DET |
|-----------------|---------|-------------------|-----------------|------|---------------|
| Row             | State ▼ | Total_order_price | Avg_order_price |      |               |
| 1               | PB      | 115268.08         | 191.48          |      |               |
| 2               | AL      | 80314.81          | 180.89          |      |               |
| 3               | AC      | 15982.95          | 173.73          |      |               |
| 4               | RO      | 46140.64          | 165.97          |      |               |
| 5               | PA      | 178947.81         | 165.69          |      |               |
| 6               | AP      | 13474.3           | 164.32          |      |               |
| 7               | PI      | 86914.08          | 160.36          |      |               |
| 8               | TO      | 49621.74          | 157.53          |      |               |
| 9               | RN      | 83034.98          | 156.97          |      |               |
| 10              | CE      | 227254.71         | 153.76          |      |               |
| 11              | SE      | 58920.85          | 153.04          |      |               |
| 12              | RR      | 7829.43           | 150.57          |      |               |
| 13              | MT      | 156453.53         | 148.3           |      |               |

Insights:

- We can see with 191.48 PB has the highest average order price, having 140 to 150 has the average order price throughout Brazil.
- There are few states like AP and RR where they have higher average order price even though having lesser total order price, which means the price per product is higher in these states.

Recommendations:

- When a customer adds a product to cart we can provide associated recommendations to increase the order price.
- Combo offers like Buy 2 get 1 etc. can be tried.

11. Calculate the Total & Average value of order freight for each state.

Answer:

*Query:*

```
SELECT
    customer_state AS State,
    ROUND(SUM(freight_value), 2) AS Total_freight_price,
    ROUND(AVG(freight_value) , 2) AS Avg_freight_price
FROM
    Target_Ecommerce.order_items o1
JOIN
    Target_Ecommerce.orders o2 ON o1.order_id = o2.order_id
JOIN
    Target_Ecommerce.customers c ON o2.customer_id = c.customer_id
GROUP BY
    State
ORDER BY
    State ;
```

*Output:*

| JOB INFORMATION |         | RESULTS             | CHART               | JSON | EXECUTION DI |
|-----------------|---------|---------------------|---------------------|------|--------------|
| Row             | State ▼ | Total_freight_price | Avg_freight_price ▼ |      |              |
| 1               | AC      | 3686.75             | 40.07               |      |              |
| 2               | AL      | 15914.59            | 35.84               |      |              |
| 3               | AM      | 5478.89             | 33.21               |      |              |
| 4               | AP      | 2788.5              | 34.01               |      |              |
| 5               | BA      | 100156.68           | 26.36               |      |              |
| 6               | CE      | 48351.59            | 32.71               |      |              |
| 7               | DF      | 50625.5             | 21.04               |      |              |
| 8               | ES      | 49764.6             | 22.06               |      |              |
| 9               | GO      | 53114.98            | 22.77               |      |              |
| 10              | MA      | 31523.77            | 38.26               |      |              |
| 11              | MG      | 270853.46           | 20.63               |      |              |
| 12              | MS      | 19144.03            | 23.37               |      |              |
| 13              | MT      | 29715.43            | 28.17               |      |              |

*Insights:*

- As the number of orders are higher in SP, RJ, MG, RS and PR the total freight price is also higher

*Recommendations:* NA

---

## Analysis based on sales, freight and delivery time.

12. Find the no. of days taken to deliver each order from the order's purchase date as delivery time. Also, calculate the difference (in days) between the estimated & actual delivery date of an order. Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- time\_to\_deliver** = order\_delivered\_customer\_date - order\_purchase\_timestamp
- diff\_estimated\_delivery** = order\_delivered\_customer\_date - order\_estimated\_delivery\_date

Answer:

*Query:*

```
SELECT
    order_id,
    order_purchase_timestamp AS Purchased_Date,
    order_estimated_delivery_date AS Expected_Date,
    order_delivered_customer_date AS Delivered_Date,
    DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp, DAY) AS Deliver_time,
    DATE_DIFF(order_delivered_customer_date , order_estimated_delivery_date, DAY) AS
    Diff_estimated_delivery
FROM
    Target_Ecommerce.orders
WHERE
    order_status = 'delivered'
Order by
    Purchased_Date DESC
```

*Output:*

| JOB INFORMATION |                               | RESULTS                 | CHART                   | JSON                    | EXECUTION DETAILS | EXECUTION GRAPH         |  |  |
|-----------------|-------------------------------|-------------------------|-------------------------|-------------------------|-------------------|-------------------------|--|--|
| Row             | order_id                      | Purchased_Date          | Expected_Date           | Delivered_Date          | Deliver_time      | Diff_estimated_delivery |  |  |
| 1               | 35a972d7f8436f405b56e36ad...  | 2018-08-29 15:00:37 UTC | 2018-09-05 00:00:00 UTC | 2018-08-30 16:23:36 UTC | 1                 | -5                      |  |  |
| 2               | 03ef5dedbe7492bdae72eec50...  | 2018-08-29 14:52:00 UTC | 2018-09-03 00:00:00 UTC | 2018-08-30 16:36:59 UTC | 1                 | -3                      |  |  |
| 3               | 168626408cb32af0ffaf76711c... | 2018-08-29 14:18:28 UTC | 2018-09-11 00:00:00 UTC | 2018-08-30 16:52:31 UTC | 1                 | -11                     |  |  |
| 4               | 0b223d92c27432930dfe407c6...  | 2018-08-29 14:18:23 UTC | 2018-09-04 00:00:00 UTC | 2018-08-30 16:24:55 UTC | 1                 | -4                      |  |  |
| 5               | 52018484704db3661b98ce83...   | 2018-08-29 12:25:59 UTC | 2018-09-03 00:00:00 UTC | 2018-08-30 22:48:27 UTC | 1                 | -3                      |  |  |
| 6               | d03ca98f59480e7e76c71fa83...  | 2018-08-29 11:06:11 UTC | 2018-09-04 00:00:00 UTC | 2018-08-30 23:56:54 UTC | 1                 | -4                      |  |  |
| 7               | d70442bc5e3cb7438da497cc6...  | 2018-08-29 10:22:35 UTC | 2018-09-04 00:00:00 UTC | 2018-08-30 16:03:19 UTC | 1                 | -4                      |  |  |
| 8               | 912859fef5a0bd5059b6d48fa7... | 2018-08-29 09:48:09 UTC | 2018-09-04 00:00:00 UTC | 2018-08-30 23:28:52 UTC | 1                 | -4                      |  |  |
| 9               | fb393211459aac00af932cd7a...  | 2018-08-29 09:14:11 UTC | 2018-09-04 00:00:00 UTC | 2018-08-30 13:03:28 UTC | 1                 | -4                      |  |  |
| 10              | bee12e8653a04e76786e8891c...  | 2018-08-29 08:46:11 UTC | 2018-09-11 00:00:00 UTC | 2018-08-30 21:54:45 UTC | 1                 | -11                     |  |  |
| 11              | c84d88553f9878bf2c7ecda2e...  | 2018-08-29 08:25:34 UTC | 2018-09-03 00:00:00 UTC | 2018-08-30 16:56:24 UTC | 1                 | -3                      |  |  |
| 12              | 4c867c2aac81653679eff07e9...  | 2018-08-28 22:51:54 UTC | 2018-08-31 00:00:00 UTC | 2018-08-31 02:03:50 UTC | 2                 | 0                       |  |  |
| 13              | e7c290bfc31d7eed478c3d3d2...  | 2018-08-28 22:30:32 UTC | 2018-09-12 00:00:00 UTC | 2018-08-30 17:18:48 UTC | 1                 | -12                     |  |  |

*Insights:*

- There are cases where the estimated delivery time is nearly 150 days but delivered in 20 days and estimated delivery time is nearly 30 days but delivered in 200 days

- As observed the recent purchases have been delivered within a week of time, sometimes within 3 days.

*Recommendations:*

- Give the estimated delivery time according to the logistics operating, on time delivery is a key factor in winning customers trust.

13. Find out the top 5 states with the highest & lowest average freight value.

Answer:

*Query:*

```

WITH
Highest AS
(
SELECT
    ROW_NUMBER() OVER(ORDER BY ROUND(AVG(freight_value) , 2) DESC ) AS SL_NO
    customer_state AS State,
    ROUND(AVG(freight_value) , 2) AS Avg_freight_price
FROM
    Target_Ecommerce.order_items o1
JOIN
    Target_Ecommerce.orders o2 ON o1.order_id = o2.order_id
JOIN
    Target_Ecommerce.customers c ON o2.customer_id = c.customer_id
GROUP BY
    State
ORDER BY
    Avg_freight_price DESC
LIMIT 5
),

```

```

Lowest AS
(
SELECT
    ROW_NUMBER() OVER(ORDER BY ROUND(AVG(freight_value) , 2) ) AS SL_NO
    customer_state AS State,
    ROUND(AVG(freight_value) , 2) AS Avg_freight_price
FROM
    Target_Ecommerce.order_items o1
JOIN
    Target_Ecommerce.orders o2 ON o1.order_id = o2.order_id
JOIN
    Target_Ecommerce.customers c ON o2.customer_id = c.customer_id
GROUP BY
    State

```

```

ORDER BY
    Avg_freight_price
LIMIT 5
)

SELECT
    *
FROM
    Highest h
JOIN
    Lowest l ON l.SL_NO = h.SL_NO
;

```

*Output:*

| Row | SL_NO | State | Highest_Avg_freight_price | SL_NO_1 | State_1 | Lowest_Avg_freight_price |
|-----|-------|-------|---------------------------|---------|---------|--------------------------|
| 1   | 1     | RR    | 42.98                     | 1       | SP      | 15.15                    |
| 2   | 2     | PB    | 42.72                     | 2       | PR      | 20.53                    |
| 3   | 3     | RO    | 41.07                     | 3       | MG      | 20.63                    |
| 4   | 4     | AC    | 40.07                     | 4       | RJ      | 20.96                    |
| 5   | 5     | PI    | 39.15                     | 5       | DF      | 21.04                    |

*Insights:*

- The average freight price varies between the range of 42 to 15.

*Recommendations:*

- We can reducing the freight price in the state having high price by having a decentralised or local inventory.
- Sourcing and supplying locally.
- By getting tied by with delivery partners.

14. Find out the top 5 states with the highest & lowest average delivery time.

Answer:

*Query:*

```

WITH
    Highest AS
    (
        SELECT
            ROW_NUMBER() OVER(ORDER BY CEILING(AVG(DATE_DIFF(order_delivered_customer_date ,
            order_purchase_timestamp, Day))) DESC) AS SL_NO,
            customer_state AS State,
            CEILING(AVG(DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp, Day))) AS
            Higest_Avg_Delivery_Time
        FROM
            Target_Ecommerce.order_items o1
    )

```

JOIN

Target\_Ecommerce.orders o2 ON o1.order\_id = o2.order\_id

JOIN

Target\_Ecommerce.customers c ON o2.customer\_id = c.customer\_id

WHERE

order\_status = 'delivered'

GROUP BY

State

ORDER BY

Higest\_Avg\_Delivery\_Time DESC

LIMIT 5

),

Lowest AS

(

SELECT

ROW\_NUMBER() OVER(ORDER BY CEILING(AVG(DATE\_DIFF(order\_delivered\_customer\_date ,  
order\_purchase\_timestamp, Day)))) AS SL\_NO,

customer\_state AS State,

CEILING(AVG(DATE\_DIFF(order\_delivered\_customer\_date , order\_purchase\_timestamp, Day))) AS

Lowest\_Avg\_Delivery\_Time

FROM

Target\_Ecommerce.order\_items o1

JOIN

Target\_Ecommerce.orders o2 ON o1.order\_id = o2.order\_id

JOIN

Target\_Ecommerce.customers c ON o2.customer\_id = c.customer\_id

WHERE

order\_status = 'delivered'

GROUP BY

State

ORDER BY

Lowest\_Avg\_Delivery\_Time LIMIT 5

)

SELECT

\*

FROM

Highest h

JOIN

Lowest l ON l.SL\_NO = h.SL\_NO

;

Output:

| JOB INFORMATION |       |       | RESULTS                  | CHART   | JSON    | EXECUTION DETAILS        | E |
|-----------------|-------|-------|--------------------------|---------|---------|--------------------------|---|
| Row             | SL_NO | State | Higest_Avg_Delivery_Time | SL_NO_1 | State_1 | Lowest_Avg_Delivery_Time |   |
| 1               | 1     | AP    | 28.0                     | 1       | SP      | 9.0                      |   |
| 2               | 2     | RR    | 28.0                     | 2       | MG      | 12.0                     |   |
| 3               | 3     | AM    | 26.0                     | 3       | PR      | 12.0                     |   |
| 4               | 4     | PA    | 24.0                     | 4       | DF      | 13.0                     |   |
| 5               | 5     | AL    | 24.0                     | 5       | GO      | 15.0                     |   |

Insights:

- The average Delivery time varies between the range of 28 days to 15 days.

Recommendations:

- Improve logistics to optimize the delivery time.
- We can reduce the time to deliver by increasing the frequency of dispatch (we can charge the customer for a faster delivery)
- Having multiple delivery partners.

15. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Answer:

Query:

```
WITH
test1 AS
(
SELECT
customer_state AS State,
CEILING(AVG(DATE_DIFF(order_delivered_customer_date , order_purchase_timestamp , DAY)))
AS Avg_Delivery_Time,
CEILING(AVG(DATE_DIFF(order_estimated_delivery_date , order_purchase_timestamp , DAY)))
AS Avg_Estimated_Delivery_Time
FROM
Target_Ecommerce.orders o1
JOIN
Target_Ecommerce.customers c ON o1.customer_id = c.customer_id
GROUP BY
State
ORDER BY
State
)
```

```

SELECT
    *,
    CEILING((Avg_Estimated_Delivery_Time - Avg_Delivery_Time)) AS Diff_Estimated_Delivery_Time
FROM
    test1
ORDER BY
    Diff_Estimated_Delivery_Time DESC
LIMIT 5;

```

*Output:*

| Row | State | Avg_Delivery_Time | Avg_Estimated_Delivery_Time | Diff_Estimated_Delivery_Time |
|-----|-------|-------------------|-----------------------------|------------------------------|
| 1   | AC    | 21.0              | 41.0                        | 20.0                         |
| 2   | RO    | 19.0              | 39.0                        | 20.0                         |
| 3   | AM    | 26.0              | 45.0                        | 19.0                         |
| 4   | AP    | 27.0              | 46.0                        | 19.0                         |
| 5   | RR    | 29.0              | 47.0                        | 18.0                         |

*Insights:*

- The difference in estimated delivery time is 20 days. This shows we are asking the customers for more time and giving the products before the asked time.

*Recommendations:*

- To reduce the difference we can discuss the time needed to deliver the product with the local delivery partner in the state and provide the estimation according.
- Sometimes customer might cancel the plan of ordering the items considering the duration to get the product.

---

### Analysis based on the payments:

16. Find the month-on-month no. of orders placed using different payment types.

Answer:

*Query:*

```

SELECT
    EXTRACT(Month FROM order_purchase_timestamp) AS Month,
    payment_type AS Mode_Of_Payment,
    COUNT(DISTINCT o.order_id) AS No_of_orders
FROM
    Target_Ecommerce.orders o
LEFT JOIN
    Target_Ecommerce.payments p ON p.order_id = o.order_id

```

GROUP BY

Month,Mode\_Of\_Payment

ORDER BY

Month, Mode\_Of\_Payment;

Output:

| JOB INFORMATION |       | RESULTS         | CHART        | JSON |
|-----------------|-------|-----------------|--------------|------|
| Row             | Month | Mode_Of_Payment | No_of_orders |      |
| 1               | 1     | UPI             | 1715         |      |
| 2               | 1     | credit_card     | 6093         |      |
| 3               | 1     | debit_card      | 118          |      |
| 4               | 1     | voucher         | 337          |      |
| 5               | 2     | UPI             | 1723         |      |
| 6               | 2     | credit_card     | 6582         |      |
| 7               | 2     | debit_card      | 82           |      |
| 8               | 2     | voucher         | 288          |      |
| 9               | 3     | UPI             | 1942         |      |
| 10              | 3     | credit_card     | 7682         |      |
| 11              | 3     | debit_card      | 109          |      |
| 12              | 3     | voucher         | 395          |      |
| 13              | 4     | UPI             | 1783         |      |

Insights:

- We can see the majority of the payments were made using credit cards.
- Number of UPI payments are less than 30% for the credit card payments.

Recommendations:

- Get in touch with banks and establish a tie-ups and provide EMI offers to the customers.
- Give day wise offers on bank credit cards, so we can build a wider association with different banks.

17. Find the no. of orders placed on the basis of the payment instalments that have been paid.

Answer:

Query:

```
SELECT
    payment_installments,
    COUNT(DISTINCT o.order_id) AS No_of_orders
FROM
    Target_Ecommerce.orders o
LEFT JOIN
    Target_Ecommerce.payments p ON p.order_id = o.order_id
WHERE
    payment_installments >= 1
GROUP BY
```

payment\_installments  
ORDER BY  
payment\_installments;

*Output:*

| JOB INFORMATION |                      | RESULTS      | CHART |
|-----------------|----------------------|--------------|-------|
| Row             | payment_installments | No_of_orders |       |
| 1               | 1                    | 49060        |       |
| 2               | 2                    | 12389        |       |
| 3               | 3                    | 10443        |       |
| 4               | 4                    | 7088         |       |
| 5               | 5                    | 5234         |       |
| 6               | 6                    | 3916         |       |
| 7               | 7                    | 1623         |       |
| 8               | 8                    | 4253         |       |
| 9               | 9                    | 644          |       |
| 10              | 10                   | 5315         |       |
| 11              | 11                   | 23           |       |
| 12              | 12                   | 133          |       |

*Insights:*

- Nearly 50% of the payments were made in instalments.

*Recommendations:*

- We can provide offers on UPI payments to increase the one shot payment, which inturn increase the cash flow.